PEMROGRAMAN SISI SERVER

Object-Relational Mapping (ORM)

Contoh dengan Python + Django



Ajib Susanto
Universitas Dian Nuswantoro

Tujuan Pembelajaran

1. Menjelaskan konsep dan manfaat ORM
2. Memetakan class Python ke tabel database dengan Django Model
3. Melakukan CRUD, filter, relasi, dan agregasi dengan Django ORM
4. Mengenali masalah N+1 dan mengoptimalkan query
5. Memahami kapan ORM cukup dan kapan perlu SQL mentah

Dunia Objek vs Relasional

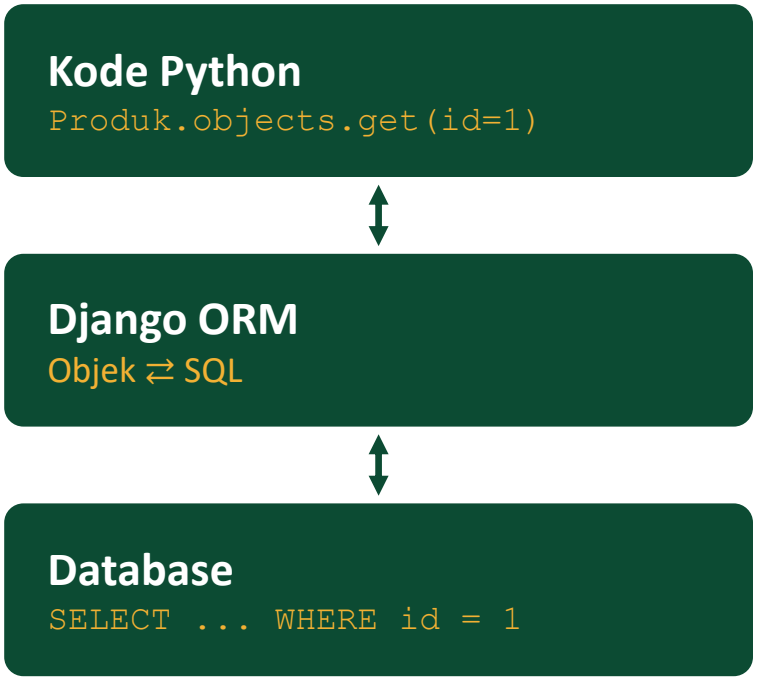
Konsep	Objek (Python)	Relasional (Database)
Struktur	Class	Tabel
Data	Atribut	Kolom
Identitas	Referensi objek	Primary key
Hubungan	Referensi / list objek	Foreign key dan JOIN
Pewarisan	Ada (inheritance)	Tidak ada secara langsung

Perbedaan ini disebut object-relational impedance mismatch. ORM menjembatani keduanya.

Apa itu ORM?

ORM (Object-Relational Mapping) adalah teknik yang memetakan objek di kode program ke tabel di database relasional.

- Developer bekerja dengan objek, bukan menulis SQL manual
- ORM menerjemahkan operasi objek menjadi query SQL
- Contoh: Django ORM, SQLAlchemy, Eloquent, Hibernate



Manfaat dan Pertimbangan

Manfaat

- Produktivitas: kode lebih singkat dan mudah dibaca
- Portabilitas: ganti database tanpa mengubah query
- Keamanan: query berparameter, mengurangi risiko SQL injection
- Konsistensi: skema dikelola lewat migrasi

Pertimbangan

- Ada overhead dibanding SQL langsung
- Mudah terjebak masalah N+1 query
- Query yang rumit kadang sulit diungkapkan
- Abstraksi bisa bocor: tetap perlu paham SQL

Active Record vs Data Mapper

Active Record

Satu class mewakili satu tabel dan sekaligus memuat operasi simpan, ubah, dan hapus.

Contoh: Django ORM, Laravel Eloquent, Rails ActiveRecord

Data Mapper

Class domain terpisah dari kode pemetaan ke database. Mapper yang menyimpan objek.

Contoh: SQLAlchemy, Doctrine, Hibernate

Django ORM memakai pendekatan Active Record: objek model menyimpan dirinya lewat p.save()

Model Django = Tabel

produk/models.py

```python
class Kategori(models.Model):
    nama = models.CharField(max_length=50)

class Produk(models.Model):
    nama = models.CharField(max_length=100)
    harga = models.PositiveIntegerField()
    stok = models.IntegerField(default=0)
    kategori = models.ForeignKey(
        Kategori,
        on_delete=models.CASCADE,
        related_name="produk",
    )
```

SQL (disederhanakan)

```sql
CREATE TABLE "produk_produk" (
  "id" integer PRIMARY KEY,
  "nama" varchar(100) NOT NULL,
  "harga" integer NOT NULL,
  "stok" integer NOT NULL,
  "kategori_id" bigint NOT NULL
    REFERENCES "produk_kategori" ("id")
);
```

Class → Tabel

Field → Kolom

Instance → Baris

Tipe Field dan Opsi

Field Django	Tipe SQL
`CharField`	VARCHAR
`TextField`	TEXT
`IntegerField`	INTEGER
`DecimalField`	DECIMAL
`BooleanField`	BOOLEAN
`DateTimeField`	TIMESTAMP
`ForeignKey`	Kolom *_id + FK

`null=True`	Kolom boleh NULL
`blank=True`	Boleh kosong di form
`default=0`	Nilai bawaan
`unique=True`	Nilai tidak boleh sama
`choices=[...]`	Pilihan terbatas

Relasi Antar Model

One-to-Many

`ForeignKey`

```
kategori = models.ForeignKey(
    Kategori,
    on_delete=models.CASCADE)
```

Satu kategori punya banyak produk. Kolom kategori_id ada di tabel produk.

Many-to-Many

`ManyToManyField`

```
tag = models.ManyToManyField(
    Tag)
```

Produk punya banyak tag, dan sebaliknya. Django membuat tabel perantara.

One-to-One

`OneToOneField`

```
akun = models.OneToOneField(
    User,
    on_delete=models.CASCADE)
```

Satu akun punya satu profil. Kolom user bersifat unik.

Migrasi: Model ke Skema

```
# Buat file migrasi dari perubahan model
python manage.py makemigrations

# Lihat SQL yang akan dijalankan
python manage.py sqlmigrate produk 0001

# Terapkan ke database
python manage.py migrate
```

migrations/0001_initial.py
File Python berisi langkah perubahan skema

sqlmigrate
Menampilkan SQL tanpa menjalankannya

django_migrations
Tabel pencatat migrasi yang sudah diterapkan

Create: Menyimpan Data

Python

```python
k = Kategori.objects.create(nama="Minuman")

p = Produk(
    nama="Kopi", harga=15000,
    stok=20, kategori=k,
)
p.save()
```

SQL setara

```sql
INSERT INTO produk_kategori (nama)
VALUES ('Minuman');

INSERT INTO produk_produk
  (nama, harga, stok, kategori_id)
VALUES ('Kopi', 15000, 20, 1);
```

`create()`
Buat dan simpan langsung

`save()`
Simpan objek yang sudah dibuat

`bulk_create()`
Banyak baris sekaligus

Read: Mengambil Data

Metode	Hasil
`all()`	Semua baris
`get()`	Tepat satu objek
`filter()`	Baris yang cocok
`exclude()`	Baris selain itu
`order_by()`	Urutan hasil
`count()`	Jumlah baris
`exists()`	True / False

```
Produk.objects.all()
Produk.objects.get(id=1)
Produk.objects.filter(stok__gt=0)
Produk.objects.exclude(stok=0)
Produk.objects.order_by("-harga")
Produk.objects.filter(stok=0).count()
Produk.objects.filter(nama="Kopi").exists()
```

Awalan minus `"-harga"` berarti urutan menurun (DESC).

QuerySet itu Lazy

```
qs = Produk.objects.filter(stok__gt=0)
# belum ada query ke database

qs = qs.order_by("nama")
# masih belum

for p in qs:    # query dijalankan di sini
    print(p.nama)
```

Lazy
Query dijalankan saat hasil dibutuhkan

Chainable
Filter bisa dirangkai berulang kali

Cache
Hasil disimpan setelah evaluasi pertama

Filter dan Field Lookup

Lookup	SQL setara
`__exact`	`=`
`__icontains`	`LIKE '%x%'`
`__gt / __lt`	`> / <`
`__in`	`IN (...)`
`__startswith`	`LIKE 'x%'`
`__range`	`BETWEEN`
`__isnull`	`IS NULL`

```
Produk.objects.filter(
    nama__icontains="kopi",
    harga__lt=20000,
)

from django.db.models import Q
Produk.objects.filter(
    Q(harga__lt=10000) | Q(stok=0)
)
```

AND otomatis antar argumen; **Q()** untuk OR dan NOT.

Update dan Delete

Update

```
from django.db.models import F

# Satu objek
p = Produk.objects.get(id=1)
p.harga = 17000
p.save()

# Massal (1 query UPDATE)
Produk.objects.filter(stok=0).update(stok=10)

# Hitung di database
Produk.objects.update(stok=F("stok") - 1)
```

Delete

```
# Satu objek
p.delete()

# Massal
Produk.objects.filter(
    stok=0).delete()
```

on_delete=CASCADE
Menghapus kategori ikut menghapus semua produknya.

Query Lintas Relasi

Python

```python
# Maju: Produk -> Kategori (JOIN)
Produk.objects.filter(
    kategori__nama="Minuman")

# Mundur: Kategori -> Produk
k = Kategori.objects.get(nama="Minuman")
k.produk.all()

# Atribut relasi
p.kategori.nama
```

SQL setara

```sql
SELECT p.*
FROM produk_produk p
INNER JOIN produk_kategori k
  ON p.kategori_id = k.id
WHERE k.nama = 'Minuman';
```

related_name="produk" membuat k.produk tersedia dari sisi Kategori.

Agregasi dan Anotasi

Python

```python
from django.db.models import Count, Sum, Avg

# aggregate: satu hasil (dict)
Produk.objects.aggregate(
    total=Sum("stok"), rata=Avg("harga"))

# annotate: nilai per objek
Kategori.objects.annotate(
    jumlah=Count("produk"))
```

SQL setara

```sql
SELECT SUM(stok), AVG(harga)
FROM produk_produk;

SELECT k.*, COUNT(p.id)
FROM produk_kategori k
LEFT JOIN produk_produk p
  ON p.kategori_id = k.id
GROUP BY k.id;
```

`aggregate()` = ringkasan keseluruhan; **`annotate()`** = nilai per baris.

Masalah N+1 Query

Tidak efisien: 1 + N query

```
for p in Produk.objects.all():
    print(p.kategori.nama)

# 1 query produk
# + 1 query kategori per produk
```

Efisien: 1 query (JOIN)

```
qs = Produk.objects.select_related(
    "kategori")
for p in qs:
    print(p.kategori.nama)

# 1 query dengan JOIN
```

`select_related`
ForeignKey dan OneToOne: JOIN, 1 query

`prefetch_related`
Relasi balik dan M2M: query terpisah, digabung di Python

Melihat SQL dan SQL Mentah

Melihat SQL

```
qs = Produk.objects.filter(stok__gt=0)
print(qs.query)       # lihat SQL
print(qs.explain())   # rencana eksekusi

from django.db import connection
print(len(connection.queries))   # DEBUG=True
```

SQL mentah

```
Produk.objects.raw(
    "SELECT * FROM produk_produk "
    "WHERE harga > %s", [10000])
```

connection.queries hanya aktif saat DEBUG=True. Untuk analisis lebih mudah, pakai django-debug-toolbar.

Awas SQL injection

Pakai parameter %s, jangan f-string atau penggabungan teks.

Transaksi dengan atomic()

```
from django.db import transaction

with transaction.atomic():
    p = Produk.objects.select_for_update().get(id=1)
    p.stok -= 1
    p.save()
    # error di sini -> semua dibatalkan
```

`atomic()`
Semua berhasil atau semua batal

`select_for_update()`
Mengunci baris; tidak berlaku di SQLite

`ATOMIC_REQUESTS`
Setting: bungkus tiap request dalam transaksi

Contoh: ORM di View

produk/views.py

```
from django.shortcuts import render
from .models import Produk

def daftar(request):
    produk = (
        Produk.objects
        .select_related("kategori")
        .filter(stok__gt=0)
        .order_by("nama")
    )
    return render(request, "produk/daftar.html",
                  {"produk": produk})
```

daftar.html

```
<ul>
{% for p in produk %}
  <li>{{ p.nama }} -
    {{ p.kategori.nama }}</li>
{% endfor %}
</ul>
```

Satu query JOIN berkat select_related; template mengakses relasi tanpa query tambahan.

Latihan Praktikum

1. Buat model Kategori dan Produk dengan ForeignKey, lalu migrasi
2. Isi data lewat shell: 3 kategori dan 10 produk
3. Tulis query: produk stok > 0 urut harga, dan jumlah produk per kategori
4. Bandingkan jumlah query dengan dan tanpa select_related
5. Tampilkan hasilnya di view dan template

Referensi

docs.djangoproject.com

Making queries dan QuerySet API reference

Ringkasan



ORM memetakan class ke tabel, atribut ke kolom, objek ke baris



Django ORM: Model, Manager, dan QuerySet yang lazy



Migrasi menjaga skema database sejalan dengan model



Filter, relasi (__), dan agregasi tanpa menulis SQL



Waspadai N+1: gunakan select_related dan prefetch_related



SQL mentah hanya bila perlu, selalu dengan parameter

Terima kasih. Ada pertanyaan?